

Environment Variable Reference

Every variable in `.env`, what it does, and whether you need it. Verified against the code on 2026-07-09 — not copied from the comments in `.env.example`, several of which are out of date.

Why does this file exist at all?

Three separate reasons, and it helps to keep them apart in your head.

1. Secrets must not live in code. Your OpenAI key is worth real money. If it were written in a `.py` file, it would be committed to git, pushed to GitHub, and scraped by a bot within hours. So the code says "read the key from the environment," and `.env` supplies it. `.env` is listed in `.gitignore`, so git refuses to track it. `.env.example` is the committed *template* — same variable names, no real values — so a new person knows what to fill in. That template/real-file split is the whole trick.

2. The same code runs in different places. The identical `server/` folder runs on your MacBook and on the Raspberry Pi. They need different settings: the robot's IP, which port to bind, whether logs should be human-readable or JSON. Rather than maintain two copies of the code, we keep one copy and two `.env` files.

3. Behavior you want to change without editing code. Which LLM handles chat, how long the mic waits before reopening, whether the camera is on by default. Changing `.env` and restarting is faster and safer than editing Python.

How it physically works

`server/config.py` runs `load_dotenv()` at import. That reads `.env` and stuffs every line into the process's environment. Then each line like:

```
CHAT_MODEL = os.environ.get("CHAT_MODEL", "gpt-4.1-nano")
```

says: *use the value from `.env`; if it's absent, use `gpt-4.1-nano`*. So most variables are optional — `.env` only overrides defaults. The rest of the code imports `config` and reads `config.CHAT_MODEL`, never the environment directly.

There is exactly **one** exception, and it's the reason a missing key produces an ugly crash instead of a helpful message:

```
OPENAI_API_KEY = os.environ["OPENAI_API_KEY"] # server/config.py:17
```

Square brackets, not `.get()`. No default. Miss it and the server dies with a `KeyError` before it prints anything useful.

The short version

If you only read one thing:

You must set	Why
<code>OPENAI_API_KEY</code>	Server crashes on import without it
<code>USE_WS=1</code>	<code>.env.example</code> ships <code>0</code> , which silently breaks the robot
<code>NAO_PASSWORD</code>	Only if you're using <code>./run.sh</code> to talk to the robot

Everything else has a working default or degrades gracefully. You can leave the entire rest of the file alone.

Group 1 — Secrets

```
OPENAI_API_KEY=sk-...
DEEPGRAM_API_KEY=
ELEVENLABS_API_KEY=
ANTHROPIC_API_KEY=
NAO_PASSWORD=
NAO_SHARED_SECRET=
```

OPENAI_API_KEY — Required. Pays for four different things: the LLM agents that decide what to say, Whisper speech-to-text (when Deepgram is off), OpenAI TTS (when ElevenLabs is off), and GPT-4o vision when someone asks "what am I wearing?". Every conversation costs money. The `nano` models are chosen throughout precisely because they're cheap.

DEEPGRAM_API_KEY — Optional. Deepgram Nova-2 is a faster speech-to-text service than Whisper. Blank means the code falls back to Whisper, which works fine and is slightly slower.

Blank, not a placeholder. `USE_DEEPGRAM` is computed as `bool(DEEPGRAM_API_KEY)` and `os.environ.get("USE_DEEPGRAM", "1") == "1"`. The string `PASTE_DEEPGRAM_KEY_HERE` is non-empty, so `bool()` is `True` — the server would try to authenticate with that literal text and the fallback would never trigger. **An unfilled placeholder is worse than an empty value.** This applies to every optional key here.

ELEVENLABS_API_KEY — Optional. ElevenLabs Flash produces the robot's voice in ~150–300 ms versus ~1–2 s for OpenAI TTS. That difference is very noticeable in conversation. Blank falls back to OpenAI TTS.

ANTHROPIC_API_KEY — Optional, almost certainly unused. Only read when `SAGE_SAFETY_PROVIDER=c laude`, which is a research setting. Leave blank.

NAO_PASSWORD — The robot's SSH password. Read **only by** `run.sh`, never by Python. It's how the script copies code to the robot and launches `main.py` there. Leave blank for laptop-only work; `run.sh` refuses to run without it.

NAO_SHARED_SECRET — A password the robot presents when opening its WebSocket. `server/app_ws.py:_check_ws_auth()` compares it against the header `X-NAO-Secret` or a `?secret=` query param.

Blank means open mode: anyone on the network can connect to your server and run up your OpenAI bill. The server logs a warning at boot. That's fine on a laptop behind a firewall. Set it to a real random string before exposing the port anywhere. It must match on both sides — `run.sh` forwards whatever is in `.env` to the robot, so they can't drift as long as you launch through it.

Group 2 — Wiring: who talks to whom

```
NAO_IP=172.20.95.127
NAO_PORT=9559
SERVER_IP=0.0.0.0
SERVER_PORT=5050
USE_WS=1
WS_HOST=0.0.0.0
WS_PORT=5050
```

NAO_IP — The robot's address on the LAN. Press the robot's chest button and it says its own IP out loud. It's a DHCP lease and can change.

NAO_PORT — 9559 is NAOqi's port, the robot's own internal robotics middleware. This is *not* your server. Leave it alone.

SERVER_IP / WS_HOST — Which network interface the server binds to. `0.0.0.0` means "all of them," which is what lets the robot reach you across the network. `127.0.0.1` would mean "localhost only" and the robot could not connect.

SERVER_PORT / WS_PORT — 5050. Both exist for historical reasons; keep them the same.

USE_WS — the one that will bite you

```
USE_WS=1          # .env.example ships 0. Change it.
```

The project used to work over plain HTTP: the robot recorded a whole sentence, POSTed it to a Flask endpoint called `/turn`, waited, and got audio back. It was rewritten to stream continuously over a WebSocket, which is what makes sub-second replies possible.

`USE_WS` chooses between them:

- `USE_WS=1` → `run.sh` boots **uvicorn** running `server/app_ws.py` (FastAPI).
- `USE_WS=0` → `run.sh` boots **Flask** running `server/server.py`.

USE_WS=0 is broken. The robot's `nao/main.py` only imports `ws_client`. The old HTTP client, `nao/conversation.py`, is dead code that nothing imports. Set `USE_WS=0` and the server starts, the robot starts, and they never speak to each other — with no loud error. `./run.sh ws` forces `USE_WS=1` for one run.

Group 3 — Voice in (speech-to-text)

```
USE_DEEPGRAM=1
DEEPGRAM_MODEL=nova-2
DEEPGRAM_LANGUAGE=en-US
WHISPER_MODEL=gpt-4o-mini-transcribe
```

Two providers, one fallback chain: **Deepgram** (if key present and `USE_DEEPGRAM=1`) → otherwise **OpenAI Whisper** using `WHISPER_MODEL`.

`USE_DEEPGRAM=1` with a blank key still means "off" — the key check comes first. So you can leave `USE_DEEPGRAM=1` permanently and control it purely by whether the key is filled in.

```
USE_SEMANTIC_ENDPOINT=0
SEMANTIC_ENDPOINT_MODEL=gpt-4.1-nano
```

USE_SEMANTIC_ENDPOINT — When you stop talking, has the robot decided you're *done*, or are you just pausing mid-thought? Voice-activity detection only hears silence. Semantic endpointing asks a small LLM "was that a complete thought?" before replying. Off by default because it costs a model call on every pause. Turn it on if the robot keeps interrupting you.

Group 4 — Voice out (text-to-speech)

```
USE_ELEVENLABS_TTS=1
ELEVENLABS_MODEL=eleven_flash_v2_5
ELEVENLABS_OUTPUT_FORMAT=mp3_44100_64
ELEVENLABS_VOICE_GIRL=
ELEVENLABS_VOICE_MAN=
ELEVENLABS_VOICE_NEUTRAL=
ELEVENLABS_DEFAULT_PROFILE=girl

USE_OPENAI_TTS=1
OPENAI_TTS_VOICE=nova
OPENAI_TTS_MODEL=tts-1
OPENAI_TTS_GAIN_DB=16
```

Chain: **ElevenLabs** (if `USE_ELEVENLABS_TTS=1` and key present) → otherwise **OpenAI TTS**. Both flags can be `1`; ElevenLabs simply wins when it can.

The three `ELEVENLABS_VOICE_*` slots are voice IDs you paste from elevenlabs.io. They're what lets a user say "switch to a man voice" mid-conversation — the choice persists per-user in SQLite. Blank slots just mean that option isn't available.

OPENAI_TTS_GAIN_DB=16 deserves a note. NAO's speaker is quiet and OpenAI's audio is mastered low, so the server runs the MP3 through **ffmpeg** to boost it 16 dB before sending. This variable is read by `server/openai_tts.py` directly, *not* through `config.py` — you won't find it there. Related: **FFMPEG_BIN** (undocumented in `.env.example`) overrides the ffmpeg path, default `ffmpeg`.

If ffmpeg isn't installed the code logs `ffmpeg unavailable, skipping gain` and carries on. The robot just speaks quietly. It is not a crash.

Group 5 — Which brain handles which question

```
ROUTER_MODEL=gpt-4.1-nano
CHAT_MODEL=gpt-4.1-nano
CHATBOT_MODEL=gpt-4.1-mini
THERAPIST_MODEL=gpt-4.1-mini
SKILLS_MODEL=gpt-4.1-nano
CRISIS_MODEL=gpt-4.1
NANO_MAX_TOKENS=200
MINI_MAX_TOKENS=400
```

Incoming speech hits the **router**, which decides who should answer, then hands off. Each agent gets its own model, and the split is deliberate:

- **nano** for the router, casual chat, and skills — these must be *fast*. A router that takes 800 ms to decide has already ruined the conversation.
- **mini** for the therapist and the Morgan-CS chatbot — these need to actually reason, and a slightly slower reply is acceptable.
- **gpt-4.1** (full size) for `CRISIS_MODEL`. When classifying whether someone is in danger, you buy accuracy with latency, every time.

`NANO_MAX_TOKENS` / `MINI_MAX_TOKENS` cap reply length. This is a *speech* interface — nobody wants a robot reading four paragraphs at them. The caps force brevity structurally rather than by asking nicely in the prompt.

`FAST_CHAT_MAX_TOKENS` (default 80, **not in** `.env.example`) caps the casual chat lane even tighter, to roughly one or two spoken sentences.

Group 6 — Knowledge, vision, safety

```
CS_NAVIGATOR_URL=
CS_NAVIGATOR_TOKEN=
CS_NAVIGATOR_TIMEOUT_S=30
```

Where the chatbot agent looks up Morgan State CS course information. It's a separate service the team deployed to Google Cloud Run. Blank means that one agent apologizes and can't answer course questions; everything else works.

```
VISION_MODEL=gpt-4o           # not in .env.example
CAMERA_DEFAULT_ON=1           # not in .env.example
CAMERA_ANNOUNCE_TEXT="..."  # not in .env.example
```

Vision is **lazy** — the camera image is only sent to GPT-4o when you say something visual ("can you see me?"). `CAMERA_DEFAULT_ON=1` means new users start with camera consent granted, and the robot announces this out loud on the first turn using `CAMERA_ANNOUNCE_TEXT`. Users opt out by saying "stop watching me." Set `CAMERA_DEFAULT_ON=0` to flip the default to opt-in.

```
SAGE_TOPOLOGY=passthrough
SAGE_SAFETY_PROVIDER=openai
SAFETY_MODEL_OPENAI=gpt-4o
SAFETY_MODEL_CLAUDE=claude-opus-4-7
```

This is the **research layer** the project is named after (SAGE-CBT). `passthrough` means "behave normally." The other topologies (`supervisor_veto` , `debate` , `shared_pool`) are experiments in having multiple agents check each other's therapeutic responses. Leave on `passthrough`.

None of this is the crisis gate. **`server/safety.py` runs before the LLM ever sees the user's message** and returns a hardcoded 988 hotline response. It is not an agent, no model can override it, and no environment variable disables it. That is deliberate. Do not add one.

Group 7 — Timing knobs

```
TTS_CHUNK_MIN_CHARS=30
TTS_CHUNK_TIMEOUT_MS=400
MIC_GATE_GRACE_MS=200
WS_RECONNECT_BACKOFF_MS=300,600,1200,2400
```

`TTS_CHUNK_*` — The LLM streams its reply one word at a time. Waiting for the whole reply before speaking would waste a second. Instead the server chops the stream into sentences and starts synthesizing speech for sentence #1 while the model is still writing sentence #2. `MIN_CHARS` is the smallest chunk worth sending; `TIMEOUT_MS` says "the model has paused this long without finishing a sentence — speak what we have."

`MIC_GATE_GRACE_MS` — After the robot stops talking, wait this long before reopening the mic, so it doesn't hear its own speaker echo and answer itself. (This is a small part of a much bigger echo-defense story — see [DECISIONS § D8](#).)

`WS_RECONNECT_BACKOFF_MS` — If the WebSocket drops, retry after 300 ms, then 600, then 1200, then 2400, then stay at 2400. Ascending so a brief network blip recovers invisibly but a dead server isn't hammered.

Group 8 — Logging and storage

```
SESSION_DB=server/nao.db
LOG_FORMAT=console
LOG_LEVEL=INFO
```

SESSION_DB — A SQLite file, created automatically. Holds conversation history, per-user camera consent, voice preference, mood logs, and CBT thought records. Deleting it resets everyone's memory. It's gitignored.

LOG_FORMAT — `console` for human-readable during development, `json` for production (machine-parseable). `.env.example` ships `json`; I set yours to `console` because you'll be reading these logs yourself.

Variables that do nothing

I traced every variable to its consumer. These are defined but **never read by any code that runs**. Setting them has no effect. Don't waste time on them, and don't trust them to work if you flip one.

Variable	Reality
<code>CBT_MODEL</code>	<code>cbt_coach.py</code> hardcodes <code>config.THERAPIST_MODEL</code>
<code>GROUNDING_MODEL</code>	<code>grounding_coach.py</code> hardcodes <code>config.THERAPIST_MODEL</code>
<code>METRICS_ENABLED</code>	<code>/metrics</code> is mounted unconditionally in <code>app_ws.py</code>
<code>OPENAI_AGENTS_TRACE</code>	The SDK reads <code>OPENAI_AGENTS_DISABLE_TRACING</code> instead
<code>SAGE_LOG_DIR</code>	Defined in <code>config.py</code> , read by nothing
<code>IMAGE_PER_TURN</code>	Only consumed by the dead <code>nao/conversation.py</code> . On the WS path the robot snaps images on <code>session_open</code> and after each reply, regardless
<code>PROACTIVE_GREET_ENABLED</code>	Only consumed by the legacy Flask <code>server/server.py</code>
<code>GOOGLE_CLOUD_PROJECT</code> , <code>VERTEX_LOCATION</code> , <code>VERTEX_DATASTORE_ID</code>	Vertex AI Search was replaced by CS Navigator. <code>chatbot.py</code> only falls back to <code>vertex_search</code> if importing <code>cs_navigator</code> <i>fails</i> , which it doesn't. <code>vertex_search.py</code> is marked <code>DEPRECATED</code> in its own first line

So: to change the CBT coach's model, you edit `THERAPIST_MODEL` — which also changes the therapist. There is no way to set them independently today without a code change.

Variables that exist but aren't in `.env.example`

`server/config.py` reads these; the template never mentions them. Defaults apply.

Variable	Default	Purpose
<code>FAST_CHAT_MAX_TOKENS</code>	80	Tight cap on the casual chat lane
<code>VISION_MODEL</code>	<code>gpt-4o</code>	Model for reading the camera image

Variable	Default	Purpose
<code>CAMERA_DEFAULT_ON</code>	<code>1</code>	Camera consent for new users
<code>CAMERA_ANNOUNCE_TEXT</code>	<i>(sentence)</i>	Spoken camera heads-up on first turn
<code>ELEVENLABS_VOICE_MY</code>	<code>""</code>	A cloned personal voice slot
<code>USE_ELEVENLABS_STT</code>	<code>0</code>	Experimental ElevenLabs Scribe speech-to-text
<code>ELEVENLABS_STT_MODEL</code>	<code>scribe_v2_realtime</code>	—
<code>REALTIME_MODEL</code>	<code>gpt-realtime</code>	Used only by the <code>realtime_proxy.py</code> experiment
<code>REALTIME_VAD_THRESHOLD</code> / <code>_PREFIX_MS</code> / <code>_SILENCE_MS</code>	—	Same experiment
<code>FFMPEG_BIN</code>	<code>ffmpeg</code>	Path to the ffmpeg binary
<code>ENABLE_NATIVE_FILLER</code>	<code>0</code>	Read by <code>nao/ws_client.py</code> . Lets NAO's built-in child voice speak filler words. Left off — the whole point is that ElevenLabs is the only voice

Safety rules

1. **Never commit `.env`.** It's gitignored. Verify with `git check-ignore -v .env`.
2. **Never paste a key into a chat, an issue, or a commit message.** If you do, revoke it immediately at the provider — assume it's compromised the moment it leaves your machine.
3. **Blank an optional key; don't leave the `PASTE_...` placeholder.** A placeholder is a non-empty string and defeats every fallback in this file.
4. **Set `NAO_SHARED_SECRET` before exposing the port** beyond your laptop.
5. If a key leaks: rotate it at the provider. Deleting the commit does not help — it's in the git history and was scrapped seconds after you pushed.